PyBaMM

Training day 2: the model zoo, custom models, input parameters and solvers

PyBaMM Conference 2026 · Ann Arbor, Michigan · 16 October 2026

Today's session

30 min

Talk: the model zoo, writing a zoo-ready model, then faster simulations

- 01 The model zoo — find, load and test zoo models
- 02 Writing your own model — a thermal DFN, zoo-ready
- 03 Input parameters — build once, solve many times
- 04 Solvers — IDAKLU settings that matter

60 min

Self-guided notebooks in training/day_2

The PyBaMM model zoo

The new home for models built on PyBaMM

New models need a home



Merge into core?

Core grows with every paper. The PyBaMM team ends up maintaining physics it didn't write. Reviews are slow.



Keep it in your own repo?

Nobody finds it. Nothing tests it against new releases. It quietly breaks and your work stops being reusable.



The model zoo

Your model lives next to PyBaMM, is tested against every change, is documented automatically, and cites you.

What the zoo gives you

1

manifest (model.toml) is all the boilerplate you write

10

contract checks run against your model for free



Tested

Every PyBaMM pull request runs your model. A weekly matrix tests it against past releases.



Documented

A docs page and a status badge are generated from your manifest and README.



Cited

Using your model registers your citation, so print_citations() credits you.

Using a zoo model

```
import pybamm
import pybamm_model_zoo as zoo

zoo.list_models()
entry = zoo.info("LinearisedSPM")
entry.tier, entry.maintainers, entry.pybamm_requires

model = zoo.load("LinearisedSPM")()
solution = pybamm.Simulation(model).solve([0, 300])
pybamm.print_citations()
```

- LinearisedSPM is the reference entry: an SPM linearised for GITT analysis
- Metadata comes from the manifest, without importing the model
- A model with missing dependencies still lists, and fails cleanly on load

Anatomy of a zoo model

```
my_model/
├── model.toml
├── __init__.py
├── model.py
├── README.md
├── CITATION.bib
├── tests/
│   └── test_my_model.py
└── examples/
    └── run_my_model.py
```

- model.toml what it is, who maintains it, which PyBaMM versions, how to test it
- model.py the physics: a pybamm.BaseModel subclass
- README.md Summary, Usage, Validation, Citation
- CITATION.bib registered when the model is created
- tests/ tests that pin a physical result
- examples/ scripts that are executed in CI

The manifest

```
[model]
slug = "thermal_dfn"
name = "ThermalDFN"
title = "DFN with a lumped thermal energy balance"
class = "pybamm_model_zoo.thermal_dfn:ThermalDFN"
tier = "community"
pybamm_requires = ">=26.9"

[[model.maintainers]]
name = "Your Name"
github = "yourhandle"

[model.citation]
key = "You2026"

[model.tests]
solve_time = 3600
key_variables = ["Voltage [V]", "Temperature [K]"]
```

- pybamm_requires drives the compatibility matrix
- tier decides whether failures block PyBaMM merges
- [model.dependencies] declares extra packages as an optional install

The contract: tested for free

manifest › layout › import › citation › well_posed › build › solve

packaging | docs | codeowners

Model checks

Valid manifest, required files, imports, registers its citation, well posed, builds, solves with finite key variables.

Repository checks

Importable inside the zoo, docs page current, an owner in CODEOWNERS. Only for models in PyBaMM's repo.

Your tests

A known limit, an analytic solution, a conservation law or a published figure.

Two tiers



community

The default for new models. Tested on every PyBaMM pull request, advisory: a failure shows up on the badge but never blocks a PyBaMM release.



core

Adopted by the PyBaMM maintainers. Part of the merge gate: a failure blocks merges in core and in the zoo.

Your model is tested from day one, without your model ever being able to block a PyBaMM release.

Writing your own model

A DFN with a lumped thermal model, zoo-ready

A standalone model is a list of equations

```
class ThermalDFN(pybamm.lithium_ion.BaseModel):
    def __init__(self, name="Thermal DFN"):
        super().__init__(name=name)
        pybamm_model_zoo.register_citation(SLUG)

        T = pybamm.Variable("Temperature [K]")
        C = pybamm.Parameter("Cell heat capacity [J.K-1]")
        ...  # DFN: particles, electrodes, electrolyte

        heat_source = current * (ocv - voltage)
        heat_loss = h * area * (T - T_amb)
        self.rhs[T] = (heat_source - heat_loss) / C
        self.initial_conditions[T] = param.T_init
```

- Variables with domains
- Parameters filled by ParameterValues
- rhs / algebraic plus boundary and initial conditions
- variables and events for outputs and cut-offs
- New parameters need defaults in default_parameter_values

A model that runs is not a model that is right

0.01 mV

Isothermal limit: huge heat capacity, so it must match PyBaMM's BasicDFN

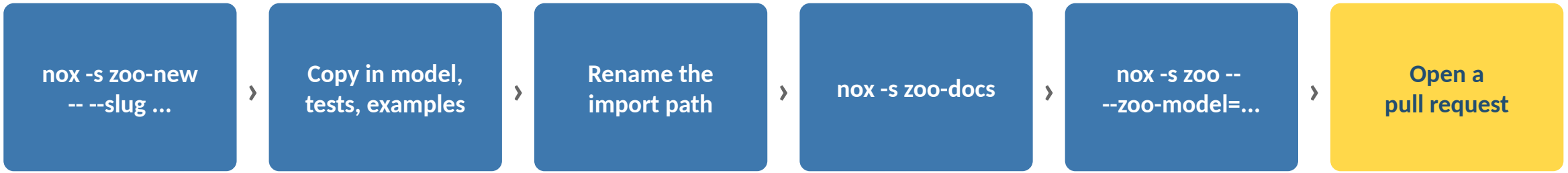
2987.5 J

Adiabatic limit: heat stored, $C \cdot \Delta T$, equals heat generated, $\int Q\,dt$ (2987.4 J)

- Limits where the answer is known are the cheapest strong tests
- Compare with the built-in DFN with lumped thermal: what does reversible heat change?

Work on it anywhere, then contribute

```
zoo.refresh(external_paths=[Path("custom_model")])
zoo.load("ThermalDFN")
```



- zoo-new renders the template and adds you to CODEOWNERS
- Zoo pull requests never touch PyBaMM core: add a line to the zoo's CHANGELOG.md
- Keeping models in your own package? Register them with the pybamm_zoo_models entry point

Faster simulations

Input parameters and the IDAKLU solver

Input parameters: build once, solve many times

```
parameter_values["Current function [A]"] = "[input]"
sim = pybamm.Simulation(model, parameter_values=parameter_values)
for current in currents:
    inputs = {"Current function [A]": current}
    sim.solve([0, 3600], inputs=inputs)
```

~12×

faster per solve for an SPM sweep: 0.050 s rebuilding vs 0.004 s with an input

Works for

Currents, diffusivities, kinetics, temperatures: anything not used to build the mesh.

Doesn't work for

Geometric parameters such as electrode thicknesses, which the default mesh needs as numbers.

IDAKLU: the settings that matter

t_eval vs t_interp

t_eval stops and restarts the solver. t_interp just samples output.

rtol, atol

Defaults 1e-4 and 1e-6. Check with a convergence study.

output_variables

Compute only the outputs you need: less memory, less time.

Parallel inputs

Pass a list of inputs; set num_threads to solve them together.

```
sol = sim.solve([0, 3600], inputs=inputs, calculate_sensitivities=True)
sol["Voltage [V]"].sensitivities["Negative particle diffusivity [m2.s-1]"]
```

Over to you

Open training/day_2 and start with 01-model-zoo

Self-guided session

Zoo track

01 and 02: load zoo models, then extend ThermalDFN with reversible heat and re-run the contract.

Speed track

03 and 04: input parameter sweeps, tolerances, parallel solves and sensitivities.

Bring your model

Copy thermal_dfn/, rename it, and get the contract passing. Then open a PR to the zoo.

github.com/pybamm-team/PyBaMM/tree/main/packages/pybamm-model-zoo